

ID: 2006A3P040PNAME: Vinay Chamola

BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE, PILANI

Second Semester 2022-23

(Part-A, CLOSED BOOK)

ECE/EEE/CS/INSTR F241 Microprocessors and Interfacing

May 15, 2023

COMPREHENSIVE EXAMINATION - PART A

60 M,

75 min

- There are three parts in this exam. PART A, Part B and Part C
- Once you turn in PART A after 75 min (strictly), PART B and Part C would be given.
- **TAKE ALL DON'T CARE VALUES STRICTLY AS 0.**
- Write all register values and memory locations/addresses in hexadecimal only.
- If any data is missing, make and state suitable assumption.

Q1. For 80386 processor working in protected memory mode of operation with paging enabled, the GDTR, LDTR, DS and CR3 value are given below:

GDTR - 00100050H, LDTR - 0030H, DS: 0026H, CR3: 7F 00 00 00H [14M]

If the instruction MOV EBX, [00 00 00 10H] is executed by the application, answer the following questions based on this information: (Refer to the GDT, LDT, PD and PT tables in the cheat sheet)

a. Which of the tables will be accessed to evaluate the final descriptor (GDT1/GDT2/LDT1/LDT2)?

LDT1

[1M]

b. Which entry number in the suitable table will be accessed to find the final descriptor for evaluating the linear address (consider the entry at base address of the table as 0th number entry)

4th

Marks will be given for 5th if a) and f) are correct

[1M]

c. What are the privilege levels of the segment which this application can access? 10, 11 [1M]

d. Which address of the GDT table will be accessed in the course of evaluating the descriptor? 100080H [2M]

e. What is the final descriptor value for evaluating the linear address? 000FB27B 0000003FF [2M]

f. What will be the linear address for this memory access? 007B0010H 00, 01 [4M]

g. What are the privilege level(s) of the application which can access this segment? 00 [1M]

h. What is the type of segment? Data - Expands upward write segment [2M]

Data segment

Q2. For 80386 processor working in protected memory mode of operation with paging enabled, the GDTR, LDTR, DS and CR3 value are given below:

GDTR - 00200000H, LDTR - 0010H, DS: 0049H, CR3: FF 00 00 00H [10M]

If the instruction MOV ECX, [00 00 00 65H] is executed by the application, answer the following questions based on this information: (Please refer to the GDT, LDT, PD and PT tables attached)

a. Which of the tables will be accessed to evaluate the final descriptor (GDT1/GDT2/LDT1/LDT2)?

GDT2

[1M]

b. Which entry number in the suitable table will be accessed to find the final descriptor for evaluating the linear address. (Consider the entry at base address of the table as 0th number entry)

9th

01809065H

[1M]

c) What is the linear address for this access? 01800065H [2M]

d) Which PD entry (e.g. 0th, 1st, ...?) is accessed for calculating the physical address? 6th [2M]

e) Which PT entry (e.g. 0th, 1st, ...?) is accessed for calculating the physical address? 9th [2M]

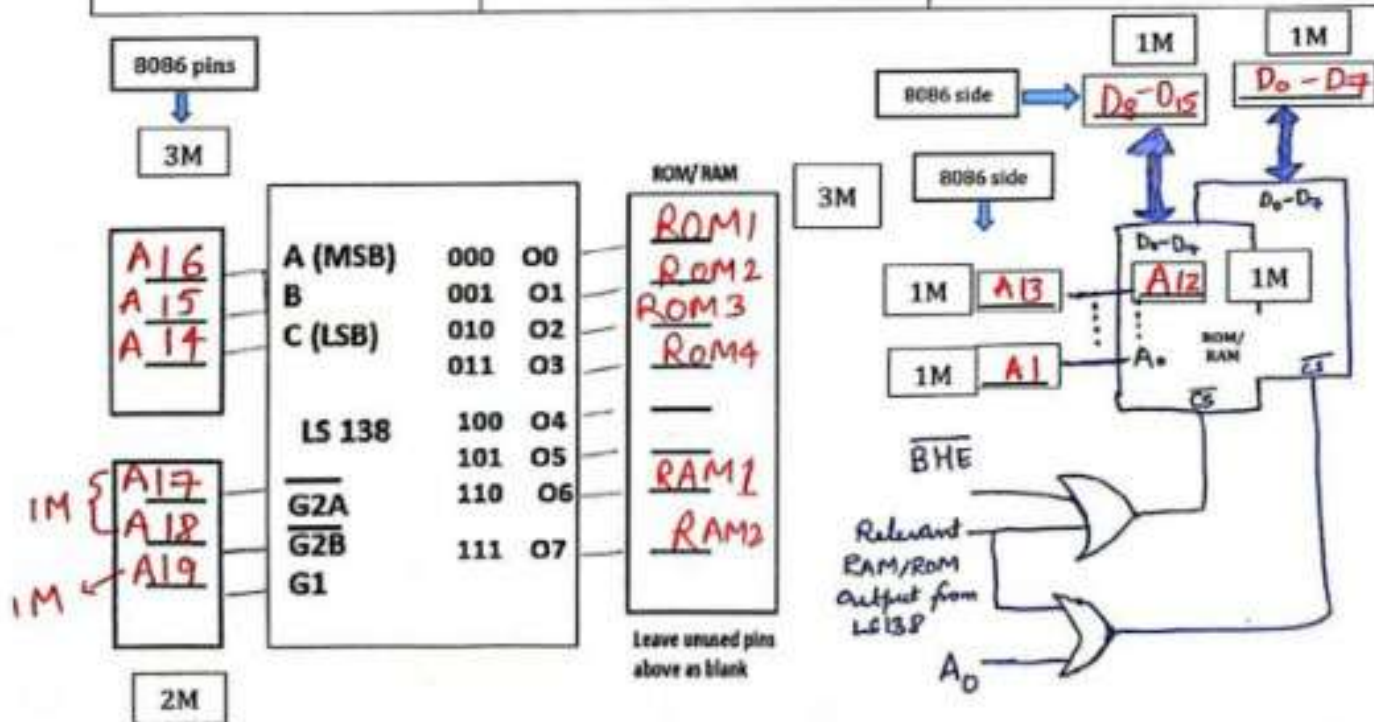
f) What is the physical address accessed? 2C000065h [2M]

Q3. (IMP: Show rough work on the right side for this question for the solution to be evaluated)
Design the memory interfacing for an 8086 based system which uses Eight 2764 ROM chips and Four 6164 RAM chips. All these chips provided should be used for interfacing. The starting address of ROM is 80000H whereas the starting address of RAM is 98000H. For the RAM/ROM interfacing, one LS138, and twelve 2-input OR gates are available. Use **absolute addressing**. [18M]

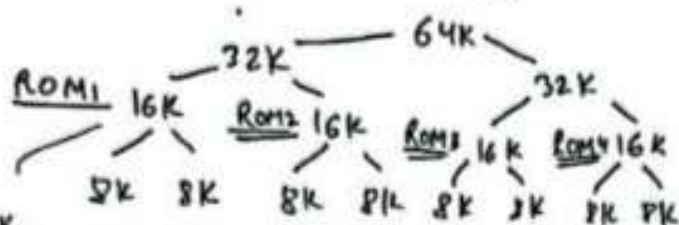
- Show the address mapping for all the memory chips by clearly filling the entries in the table below. [5M]
- Fill in the blanks in the memory interfacing shown below the table. [13M]

Answer a. Use only that many rows as required (do not make separate entries for odd/even ROM/RAMs)

RAM/ ROM (e.g. RAM 1 etc.)	Starting address (H)	Ending address (H)
1M — ROM 1	80000H	83FFF
1M — ROM 2	84000	87FFF
1M { ROM 3	88000	8BFFF
1M { ROM 4	8C000	8FFFF
1M — RAM 1	98000	9BFFF
1M — RAM 2	9C000	9FFFF

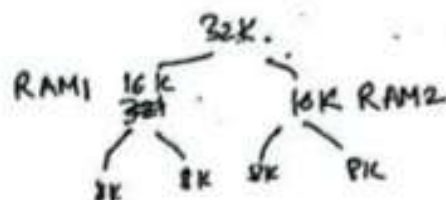


Kindly show rough work here for Q3



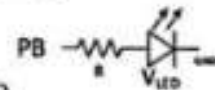
Stack addr = 80000

11

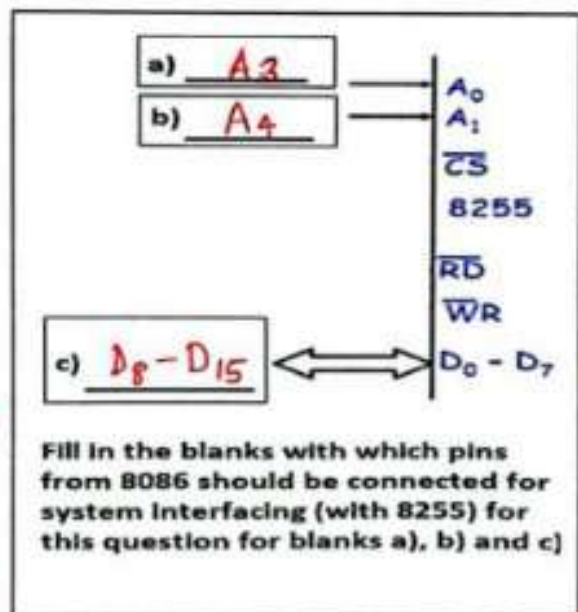

$$\begin{array}{r} 16K \\ 2^{10} \times 2^4 \\ \hline 2^{14} \end{array}$$

		A0	A1	A2	A3	A4	A5	A6	A7	A8	A9	A10	A11	A12	A13	A14	A15	A16	A17	A18	A19	A20
RAM1	{	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
		1	0	0	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
RAM2	{	1	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
		1	0	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
RAM3	{	1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
		1	0	0	0	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
RAM4	{	1	0	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
		1	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
RAM1	{	1	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
		1	0	0	1	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
RAM2	{	1	0	0	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
		1	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

Q4. There is an 8086 based **drone system** to be designed which uses 8255 for I/O purpose. Port A and Port C of the 8255 are used to interface various sensor inputs. Port B of 8255 has 3 LEDs connected which indicate which mode the drone is currently in i.e. *bad weather mode* (red LED glows), *landing mode* (yellow LED glows) and normal *flight mode* (green LED glows). Only one of these LEDs glows at a given point of time. These 3 LEDs are connected to **PB3 (red)**, **PB4 (yellow)** and **PB5 (green)**, respectively (Note: Port B pins are numbered from PB0 to PB7). The base address of the 8255 is **E7H**. Assume that there is a procedure **status_check** which analyzes sensor inputs (from Port A and Port C) and indicates in which mode the drone is operating. This procedure stores a particular value in BL register based on which mode the drone is operating in. The value stored in the BL register is 0H, 1H and 2H for the *bad weather mode*, *landing mode* and *flight mode*, respectively. Complete the following code which helps in the functionality of the LED glowing for the drone as per above scheme. Also fill in the blanks in the figure below [18M].



LED's are connected to port B in this configuration (as shown in the figure)



PortA EQU E7H

PortB EQU EFH

PortC EQU F7H

CREG EQU FFH

MOV AL, 99H

OUT CREG, AL

CALL *status_check*

CMP BL, 0

JE *bad_weather_mode*

CMP BL, 1

JE *landing_mode*

CMP BL, 2

JE *normal_flight_mode*

bad_weather_mode: mov al, 08h — 1.5M

out portB, al

landing_mode: mov al, 10h — 1.5M

out portB, al

flight_mode: mov al, 20h — 1.5M

out portB, al

1.5M

→ All other fill in the blanks except this are 1M each

ID: _____ NAME: _____

BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE, PILANI
Second Semester 2022-23 (Part-B OPEN BOOK)
ECE/EEE/CS/INSTR F241 Microprocessors and Interfacing
May 15, 2023 COMPREHENSIVE EXAMINATION - PART B 40 M, ~ 60 MIN

- TAKE ALL DON'T CARE VALUES STRICTLY AS 0.
- Write all register values and memory locations/addresses in hexadecimal only.
- If any data is missing, make and state suitable assumption.

Space for recheck request:

Q1. Consider the following 8086 code snippet used to generate a software delay. [Total Marks for this question- 6M]

MOV CX, VAL ; instruction 0. VAL is an immediate data

X1: AND AX, AX ; instruction 1

OR BX, BX; instruction 2

NOP; instruction 3

LOOP X1; instruction 4

- a) What is the number of clock cycles taken by 8086 to execute from instruction 1 to instruction 4 if CX value after the execution of instruction 4 is a non-zero value? [1M]

Ans: 26 cycles

- b) What is the value of VAL that must be written into CX IN HEXADECIMAL FORMAT if instruction 1 to instruction 4 should generate a delay of 200 milli seconds? Assume that 8086 is connected to a 4 MHz clock? [3M]

Ans: VAL= 7831H or 7832H H (30769 OR 30770 in decimal)

Show your working below on how you obtained the value of VAL

A) 4 MHz => 250ns

Ans: AND, NOP and NOR instruction → 3 clocks each

$$[(9+17)*VAL-12]*250*10^{-9} = 200 \times 10^{-3}$$

$$\Rightarrow VAL = 30769.69 \sim 30769 \text{ OR } 30770$$

- c) If CX is loaded with the value calculated by you in part-b, what is the delay generated (accurate upto 7 decimals without rounding off) which is actually generated by this code? Assume that there are no other causes of delays in this system. [2M]

Ans: 0.200002000 or 0.199995500 sec

Show your working below on how you obtained the above answer:

$$B) 26*30769-12)*250*10^{-9} = 0.199995500 \text{ sec or}$$

$$(26*30770-12)*250*10^{-9} = 0.200002000 \text{ sec}$$

Q2. Use 8254 to generate an interrupt to 8086 once in every 85896 seconds? An inverter will be used to connect the relevant output from 8254 to the INTR pin of 8086. Assume that you have only a 4 MHz clock available. **Also Counter 0 is already loaded with FFFFH which should not be changed by you.** Now answer the following questions for this interfacing and functionality.

- a) Which counters of 8254 will be used for this scheme? Write "Used" or "Not used" against each counter in the following table. [2M]

Ans:

Counter number	Used/Not Used
Counter 0	Used
Counter 1	Used
Counter 2	Used

2 Marks if fully correct

- b) What modes are each of these counters working in? Please enter the mode against the corresponding counter. If a counter is not used put "Not used" in the mode column against the corresponding counter. [3M]

Counter number	Mode
Counter 0	Mode 2 or mode 3 →
Counter 1	Mode 2 or mode 3 →
Counter 2	Mode 2 →

1 Mark
1 Mark
1 Mark

- c) What is the count to be loaded in each of these counters? While loading the counts, the total number of write operations for writing the count into the counters must be minimized. Note that **Counter 0 is already loaded with FFFFH which should not be changed by you.** Mention the count in the following table. If a counter is not used put "Not used" in the count loaded column against the corresponding counter. [5M]

Counter number	Count Loaded in hexadecimal	Count Loaded in decimal
Counter 0	FFFFH	65535
Counter 1	FFFFH	65535
Counter 2	50h	80

3 Mark if only fully correct else zero marks

Show your working below on how you obtained the count values written in the above table:

$$85896 \text{ seconds} \Rightarrow 1.164198566 \times 10^{-5} \text{ Hz}$$

$$\text{Count required is } 4 \text{ MHz} / 1.164198566 \times 10^{-5} \text{ Hz} = 3.43584 \times 10^{11}$$

$$3.43584 \times 10^{11} = 65536 \times 65536 \times 80$$

- d) If the base address of 8254 is F9H, write a code snippet to configure the counters for the modes they are working in. Use only 6 lines for this code. Strictly use hexadecimal format only for the data or address values used in the code. [3M]

Ans:

only if correct

- 1 Mark { 1. MOV AL, 34H OR MOV AL, 36H ; CONFIGURING COUNTER 0
2. OUT FFH, AL
1 Mark { 3. MOV AL, 74H OR MOV AL, 76H ; CONFIGURING COUNTER 1
4. OUT FFH, AL
1 Mark { 5. MOV AL, 94H or MOV AL, 95H ; CONFIGURING COUNTER 2
6. OUT FFH, AL

If counter 1 loaded with 1 byte and counter 2 with 2 bytes
CW for Counter 1 = 54 H
or 55 H
CW for Counter 2 = B4 H +
or B6 H

Show your working below on how you obtained the control words:

Counter-0 0011 0100 or 0011

0110 = 34 H or 36H

Counter-1 0111 0100 or 0111 0110 =

74H or 76H

Counter-2 1001 0100 = 94H OR

1001 0101 = 95H

ADDRESS COUNTER 0- F9H 1111 1001H

COUNTER1- 1111 1011- FBH

COUNTER2- 1111 1101- FDH

CONTROL REGISTER 1111 1111- FFH

- e) If the base address of 8254 is F9H, write a code snippet to load the initial count into each of the counters used. Use only 7 lines for this code. Strictly use hexadecimal format only for the data or address values used in the code. [3 M]

MOV AL, FFH

OUT F9H, AL → optional

OUT F9H, AL → optional

OUT FBH, AL

OUT FBH, AL

MOV AL, 50H ; if 94H WAS USED IN PART

FOR CONFIGURING COUNTER 2 else this will be 80H

OUT FDH, AL

no marks for this

1.5 M

1.5 M

If CW for counter 2 is 94H in part (d) 50H must be used if CW is 95H 80H must be used

Q3. In an 8086 processor, assume that the following instructions are at even address locations. Specify what will be the type of machine cycle and the purpose of the machine cycle (i.e., Instruction Fetch/Operand Fetch/ Result WriteBack) in the sequence they are performed. [3M+2M+3M=8M]

A) XLAT

S. No.	Type of Machine Cycle	Purpose
1	MEMR	INSTRUCTION FETCH
2.	MEMR	OPERAND FETCH

Note: from the Appendix of the text book it can be seen that XLAT is a 1 byte instruction

3 marks if fully correct else zero marks

2 Marks only
if fully
correct
else 0 marks
(zero)

B) INC WORD PTR [BX][SI][0123H]

S. No.	Type of Machine Cycle	Purpose
1	MEMR	INSTRUCTION FETCH
2.	MEMR	INSTRUCTION FETCH
3.	MEMR	OPERAND FETCH
4.	MEMW	RESULT WRITEBACK

Note: This is a 4 byte instruction

3 M if
full correct
or 0 Marks
if some
mistake

C) IN AL, 78H

S. No.	Type of Machine Cycle	Purpose
1	MEMR	INSTRUCTION FETCH
2.	IOR	OPERAND FETCH

Note: This is a 2 byte instruction

Q4. An 8259 interfaced with 8086 has the base address as FFEFH. Write the ICWs to initialize the 8259 in a single level triggered mode, with call address interval of 4, non-buffered and no special fully nested mode. Then write the OCWs to set the 8259 to operate with IR0 and IR5 masked, IR2 as bottom priority level with rotate on specific EOI mode. The 8259 sends interrupt type 79 (decimal) to the 8086 in response to an interrupt request at IR6 input. Strictly treat all don't cares bit(s) as zero. Mention the ICWs and OCWs clearly first in hexadecimal format in the blanks provided below. If any ICW or OCW is not required, mark "not required" in the blank against the corresponding ICW/OCW. [10M]

Ans: ICWs

OCWs

- 2 M ← i) ICW1=1FH or 1BH → 2 Marks
1 M if some attempt to solve is done ← ii) ICW2=48H
1 M ← iii) ICW3 NOT NEEDED → 1 Mark
2 M ← iv) ICW4 01H → 2 Marks

- i) OCW1= 21H → 1 Mark
ii) OCW2= E2H → 2 Marks
iii) OCW3 NOT NEEDED → 1 Mark

Show your working for obtaining the ICWs and OCWs (no credit without proper working):

$$ICW1 = xxx1\ 1111 = (00011111)_2 = 1FH$$

ICW2

$$79 = 4Fh \text{ FOR IR6}$$

$$\Rightarrow 48H = (0010\ 1000)_2 \text{ FOR IR0}$$

$$\Rightarrow ICW2 = 48H$$

ICW3 NOT NEEDED AS SINGLE MODE

$$ICW4 = 0000\ 0001 \text{ (NO SFNM, NO BUFFERED, NOT AEOI)} \\ = 01H$$

OCW1

$$IR0 \text{ and } 5 \text{ masked so } OCW1 = (00100001)_2 = 21H$$

$$OCW2 = 1110\ 0010$$

$$R\ SL\ EOI = 111 \text{ ROTATE ON SPECIFIC EOI AND } L2\ L1\ L0 = 010 \text{ (SETS IR2 as lowest)}$$

- Answer the questions in the space provided
- No need to ask anything. All the information is given in the question paper itself. Read it carefully.

Q1. Based on the given ALP and the outputs from DOSBOX for the 'u' command and the 'd' command, (show in the figure below), identify and provide the missing information by filling the blanks below? You need to fill in the blanks provided below. **DO NOT FILL THE BLANKS IN THE FIGURE [8M]**
The ascii value of 'a' is 61h, 'A' is 41h, '0' is 30h, '[' is 5Bh, ']'5Dh and '*' is 2Ah.

```
.model tiny
```

```
.data
```

```
data1 db '[', ']', '*', '*'
```

```
data2 dw 9876h, 'a', 'A', '0'
```

```
data3 dw 0fefeh
```

```
array dw 01,02,03,04,05,06,07,08,09
```

```
.code
```

```
.startup
```

```
mov BX,data2
```

```
mov CX,data3
```

```
mov data1,BL
```

```
mov DL,data1
```

```
mov AX,array[DI]
```

```
.exit
```

(a) BX, [011C] [1.5 M]

(b) CX, [0124] [1.5 M]

(c) 0108 [1 M]

(d) 010C [1 M]

(e) 76986100 [1.5 M]

(f) 0030 00FE [1.5 M]

```

DOSBox: 0.74 Cpu speed: 3000 cycles Frameskip: 0 Program:
0063:0268 09 EB 07 FF 46 06 EB CD-2B C0 5D CB 55 0B EC 03 ....F...*.U...
0063:0270 EC 02 C7 46 FE 00 00 8B-46 FE 40 3D 06 06 57 73 ...F...F.0:...Me
0063:0280 22 8A 46 06 C4 36 00 57-8B 5E FE 26 3A 00 72 00 ".F..G.W..A:r.
0063:0290 26 3A 40 01 77 05 00 01-00 EB 0A 03         &:0.w.....

-q
D:\>debugx ex.com
-q
0063:0100 8B1E1C01      MOV     [a]
0063:0104 8B0E2401      MOV     [b]
0063:0108 0B1E1901      MOV     [0119],BL
0063:010C 0A161901      MOV     BL,[0119]
0063:0110 0B052601      MOV     AX,[DI+0126]
0063:0114 B44C          MOV     AH,4C
0063:0116 CD21      INT     21
0063:0118 005B5D      ADD     [BP+DI+5D],BL
0063:011B 2A7698      SUB     BH,[DP+60]
0063:011E 61          POPA
0063:011F 004100      ADD     [BX+DI+00],AL
-q 0119
0063:0110
0063:0120 41 [c] FE 01 00-02 00 03 00 04 00 05 00 A.0.....
0063:0130 06 00 07 00 00 00 09 00-0B E5 5D CB 55 0B EC FF .....].U...
0063:0140 76 06 9A 2D 14 EF 06 0B-E5 5D CB 55 0B EC 03 3E v...-.....].U...>
0063:0150 06 57 02 73 13 0A 46 00-2A E4 50 FF 76 06 9A 0E .U..F..F.v...
0063:0160 00 08 09 03 C4 04 EB 30-8D 76 06 0A 04 22 C0 74 .....0.v...".t
0063:0170 25 2A E4 50 0E E0 74 00-83 C4 02 23 C0 74 05 FF x.F.....s.t..
0063:0180 46 06 EB 00 0B 76 06 0A-04 3A 46 00 75 03 96 EB F...v...:F.w...
0063:0190 07 FF 46 06 EB D2 2B C0-5D          ..F...*.]
  
```

Q2. Here's an assembly language program to find the element in the array with the most number of 1's in its binary representation. This code modifies the ALP of finding greatest number in an array and adds a new subroutine called "CountOnes" to count the number of 1's in the binary representation of a byte. Fill in the missing code to complete the functionality.

Write in the blanks on the right side. (a,b,c,d,e,f - 1M each) [6M]

.mode tiny

.data

ARRAY1 db 91H, 02H, 083H, 0FFH, 075H, 06H, 47H, 12H, 76H

RESULT db ?

MAX_ONES db 0

.code

.startup

```
LEA BX, ARRAY1
MOV CL, 09H
MOV AL, [BX]
MOV DL, AL
CALL COUNT_ONES
MOV (a), AH
MOV RESULT, AL
INC BX
X2: MOV AL, [BX]
CALL COUNT_ONES
CMP AH, MAX_ONES
(b)
MOV MAX_ONES, AH
MOV RESULT, AL
SKIP: INC BX
DEC CL
(c)
.EXIT
```

```
CONT_ONES: PUSH AX
            PUSH BX
            PUSH CX
            MOV AH, 0
            MOV CL, (d)
COUNT_LOOP: SHR AL, 1
              JNC (e)
              INC AH
NO_INCREMENT: DEC CL
              (f)
              POP CX
              POP BX
              POP AX
              RET
END
```

(a) MAX_ONES

(b) JLE SKIP / JLSKIP / JS SKIP
JBSKIP / JBESKIP.

(c) JNZ X2 / JNE X2

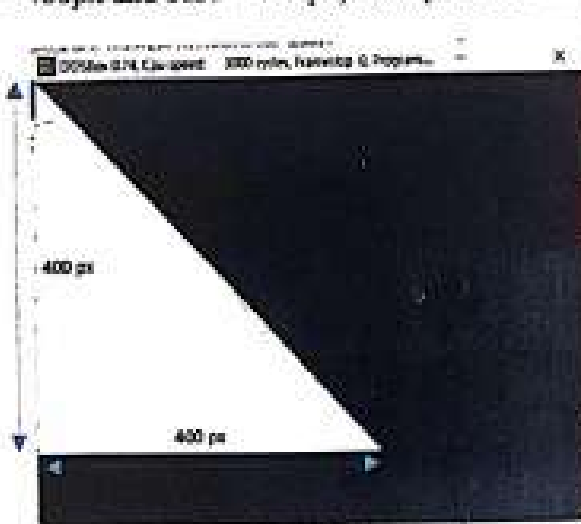
(d) 08H

(e) NO_INCREMENT

(f) JNZ COUNT_LOOP
OR JNE COUNT_LOOP

JNA skip
JNB skip

Q3. Here is a part of code snippet to print a right angled triangle of white color having given shape. (height = 400px and base = 400 px). Complete code to print this shape. (a,b,d,e - 1.5M each, c,f - 1M each) [8M]



; Paint the triangle

PAINT1:

(c) MOV SI, rowstr ; Row start
1M

ROW1:

(d) MOV CX, SI ; Number of columns in current row
1.5M

COLUMN1:

DEC CX

MOV DI, CX

PUSH CX

MOV AH, 0Ch

OR MOV AL, 00001111b

(e) MOV AL, 0FH ; Color for triangle
1.5M

MOV CX, 01

MOV DX, SI

INT 10h

POP CX

CMP CX, 0

; End of row

JNZ COLUMN1

INC SI

(f) MOV AX, rowend ; Row end
1M

CMP SI, AX

JNZ ROW1

; Initialize parameters for triangle
drawing based on diagram

MOV rowstr, (a) 1 ; 1.5M

MOV rowend, (b) 401 ; 1M

9. (a) 0 } student will be
(b) 400 } given 1.5M

(190H) } (a)+(b)

Page Total

a < b

- c) Assuming we have done the same thing for second file, and have the handle of second file stored in handle2, complete the code snippet below which reads and stores the character from both files in buffer1 and buffer2 respectively, and compares them character by character.

; Compare files

FileCompare:

; Read a character from first file

MOV AH, 3Fh
LEA DX, buffer1
MOV BX, handle1

(d) //3lines [2 M]

mov cx, 1

int 21h

cmp ax, 1 ; Check if we read 1 character

jne FilesAreDifferent

; Read a character from second file

mov ah, 3Fh

lea dx, buffer2

mov bx, handle2

MOV CX, 1

INT 21h

cmp ax, 1

jne FilesAreDifferent

(e) //4lines [2 M]

; Compare the characters

mov al, buffer1

cmp al, buffer2

JE FileCompare

If characters are equal, continue to next character

(f) //1line [1 M]

JZ FileCompare

FilesAreDifferent:

; Check the reason for leaving the loop

cmp ax, 0

je FilesAreEqual ; If reached EOF for both files, they are equal

; Display message "Files are different"

lea dx, msg4

mov ah, 09h

int 21h

jmp CloseFiles

FilesAreEqual:

; Display message "Files are identical"

LEA DX, msg3

MOV AH, 09h

INT 21h

(g) //3lines [2 M]